Prototype Formulasi Risk Scoring

Karena mengejar *prototype* untuk rentang 12 minggu dengan skala 0-100 dan grafik dinamis, kita bisa membangun model **Dynamic Risk Scoring** menggunakan formula klasik manajemen risiko:

Risiko = Dampak * (Kerentanan + Ancaman)

$$R = I * (w1*V + w2*T)$$

Parameter-parameter ini diambil dari Wazuh dan kuesioner, lalu dirumuskan ke dalam backend.

(Asset = Agent = Server).

1. Dampak / *Asset Criticality* (I), Data Internal

Ini diambil dari kuesioner skala Likert (1-5) yang diisi oleh manajemen berdasarkan kacamata *Balanced Scorecard* (seberapa penting server ini bagi finansial/operasional bank).

- **Likert 1:** Sangat tidak penting (Misal: PC resepsionis)
- **Likert 5:** Sangat penting (Misal: Server *Database* Nasabah Bank)

Kita normalisasi nilai ini agar menjadi *multiplier* (pengali) maksimal 1 (atau 100%):

(Jika skor Likert 5, maka I = 1.0. Jika skor 3, maka I = 0.6)

Skala Penilaian (Likert 1-5):

- 1 = Sangat Tidak Setuju (Dampak Sangat Rendah)
- 2 = Tidak Setuju (Dampak Rendah)
- 3 = Netral (Dampak Sedang)
- 4 = Setuju (Dampak Tinggi)
- 5 = Sangat Setuju (Dampak Sangat Kritis)

Kuesioner Penilaian Kekritisan Tiap Aset (Asset Criticality)

A. Perspektif Keuangan (Financial Perspective)

Fokus pada kerugian uang secara langsung akibat downtime atau peretasan.

1. Jika aset/sistem ini mengalami gangguan (downtime) selama lebih dari 4 jam, operasional bank akan mengalami kerugian finansial yang signifikan secara langsung.
2. Aset ini digunakan secara langsung untuk memproses, memvalidasi, atau menyimpan data transaksi keuangan dengan perputaran dana yang besar.

B. Perspektif Pelanggan & Reputasi (Customer Perspective)

Fokus pada kepercayaan nasabah dan citra bank di mata publik.

3. Kebocoran data yang ada di dalam sistem ini akan sangat merusak kepercayaan nasabah dan mencoreng reputasi bank di media.

4. Sistem ini adalah titik kontak utama layanan bank (misal: mobile banking, internet banking, atau sistem ATM) yang jika mati, nasabah akan langsung mengeluh/terdampak.

C. Perspektif Proses Internal (Internal Business Process)

Fokus pada kelancaran operasional harian karyawan dan antar-divisi.

5. Aset ini sangat esensial bagi operasional harian; tanpa sistem ini, proses kerja utama di kantor cabang atau pusat akan lumpuh total.

6. Data dan layanan pada sistem ini tidak memiliki backup manual yang cepat, sehingga staf tidak bisa menggunakan "cara manual/kertas" saat sistem ini diserang.

D. Perspektif Pembelajaran & Pertumbuhan / Kepatuhan (Learning, Growth & Compliance)

Fokus pada regulasi pemerintah (OJK/BI) dan rahasia perusahaan.

7. Sistem ini menyimpan data pribadi nasabah yang diatur sangat ketat oleh regulasi (seperti UU Pelindungan Data Pribadi / aturan OJK).

8. Kompromi atau peretasan pada sistem ini dapat mengakibatkan bank terkena sanksi hukum, denda dari regulator, atau pencabutan izin.

2. Kerentanan / *Vulnerability Score* (V), Data CIS Wazuh

Ini adalah postur keamanan statis yang diambil dari modul **SCA (Security Configuration Assessment)** di Wazuh yang mengaudit standar **CIS Controls**. Wazuh akan memberikan persentase *pass/fail*. Semakin banyak yang gagal, semakin rentan sistemnya.

V = 100 - Persentase CIS yang "Passed"

(Contoh: Jika server hanya lulus 60% standar CIS, maka nilai kerentanannya V = 40.)

A. Apa itu Wazuh SCA?

Berdasarkan dokumentasi resminya, **Security Configuration Assessment (SCA)** adalah modul bawaan Wazuh yang berfungsi untuk memverifikasi apakah *endpoint* (server/komputer yang dipantau) sudah dikonfigurasi sesuai dengan standar keamanan industri, salah satunya adalah **CIS (Center for Internet Security) Benchmarks**.

Wazuh menggunakan *policy files* (berformat YAML) yang berisi ratusan aturan (*rules*). Aturan-aturan ini secara otomatis memindai sistem untuk mencari miskonfigurasi, seperti:

- Apakah *firewall* bawaan sudah aktif?
- Apakah *password* default sudah diganti?
- Apakah hak akses folder sensitif sudah dikunci?
- Apakah *remote login* anonim (seperti SSH *root login*) sudah dimatikan?

B. Bagaimana Sistem *Pass/Fail* Bekerja?

Setiap kali agen Wazuh melakukan pemindaian (biasanya berjalan secara berkala, misalnya setiap 12 atau 24 jam), setiap aturan di dalam *policy file* akan dievaluasi dan menghasilkan salah satu dari tiga status:

- **Passed:** Konfigurasi server sudah aman dan sesuai dengan standar CIS.
- **Failed:** Ditemukan miskonfigurasi atau celah keamanan (misal: *port* terbuka yang seharusnya ditutup).
- **Not Applicable:** Aturan tersebut tidak relevan dengan sistem yang sedang dipindai (misal: aturan untuk web server Apache, tapi server tersebut tidak menggunakan Apache).

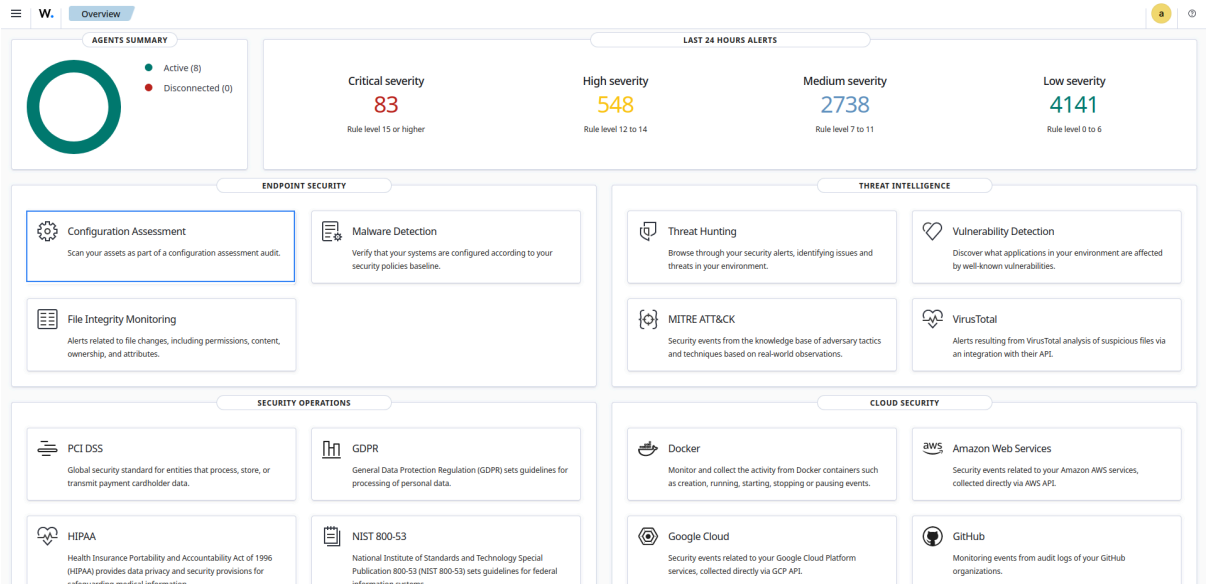
Di dalam *dashboard* Wazuh, hasil akhir dari pemindaian ini akan diakumulasikan menjadi sebuah **SCA Score** dalam bentuk persentase (0% hingga 100%). Persentase ini merepresentasikan jumlah rasio aturan yang berstatus *Passed* dibandingkan total aturan yang relevan.

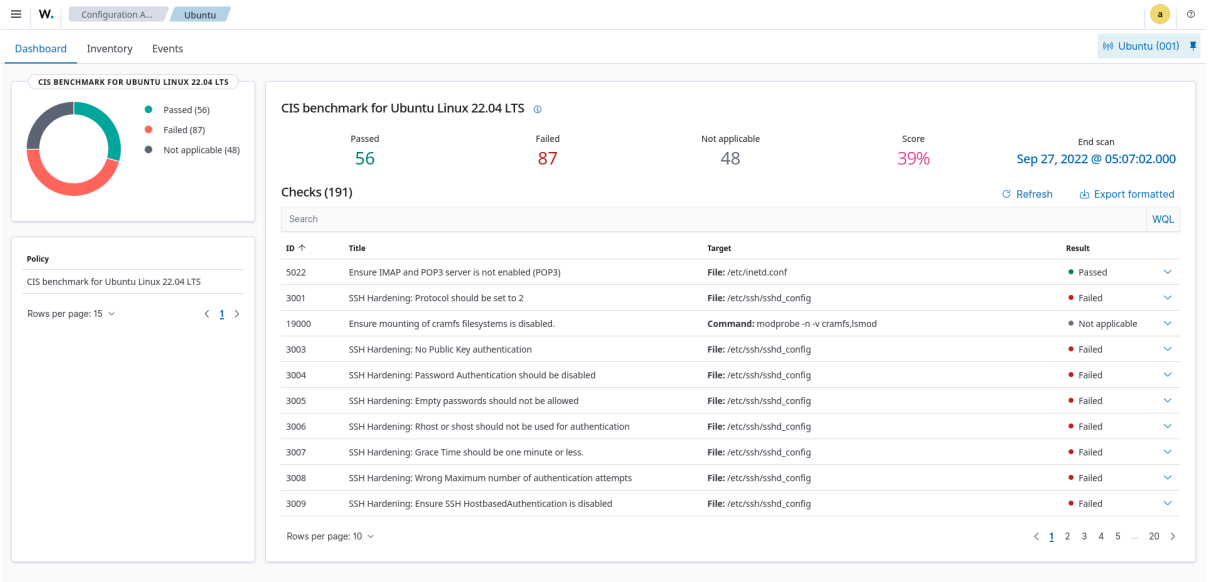
C. Implementasi ke Formula *Vulnerability Score* (V)

Karena sistem skor risiko yang dibangun (0-100) mengasumsikan bahwa angka **100 adalah tingkat risiko paling kritis/berbahaya**, maka tidak bisa mentah-mentah memasukkan *SCA Score* bawaan Wazuh.

Wazuh menggunakan logika kebalikan: *SCA Score* 100% berarti sistem 100% aman. Oleh karena itu, di sistem *backend* nanti, harus membaliknya menjadi nilai kerentanan (*vulnerability*):

V = 100 - SCA Score Wazuh





Contoh di Laporan/Dokumentasi:

"Server Windows Bank Jatim dipindai menggunakan modul SCA Wazuh dengan standar *CIS Benchmark for Windows Server*. Dari 191 pengecekan, 56 berstatus *Passed* dan 87 berstatus *Failed*. Maka *SCA Score* di Wazuh adalah **39%**. Oleh karena itu, nilai Kerentanan (V) server tersebut di dalam *Scoring Engine* adalah 100 - 39 = 61."

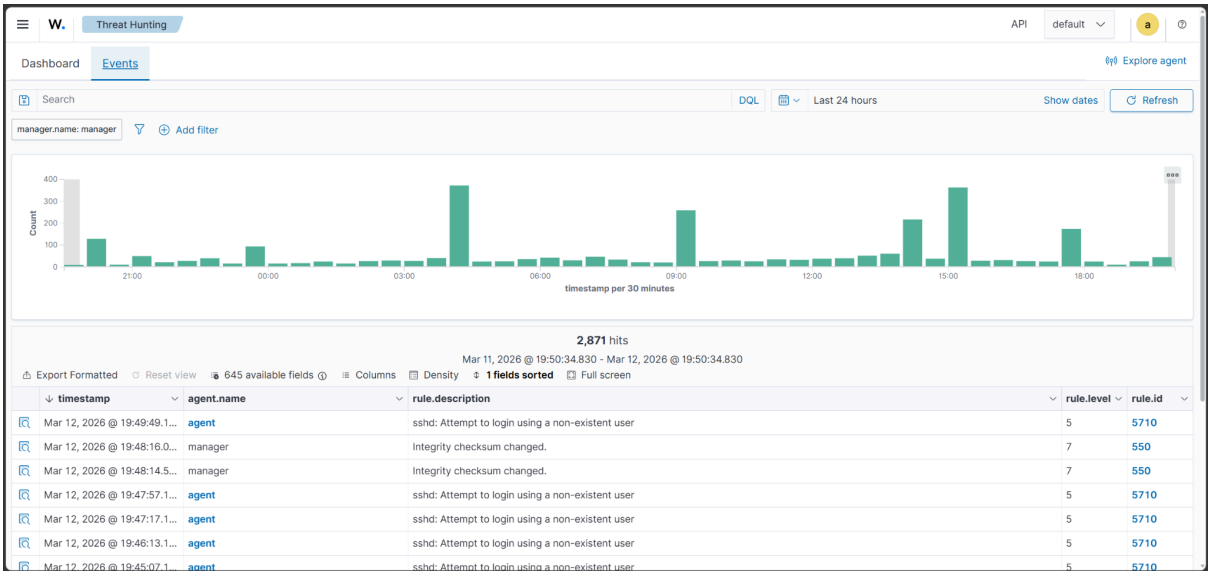
Penjelasan Konsep SCA: [Wazuh Configuration Assessment - Use Cases]

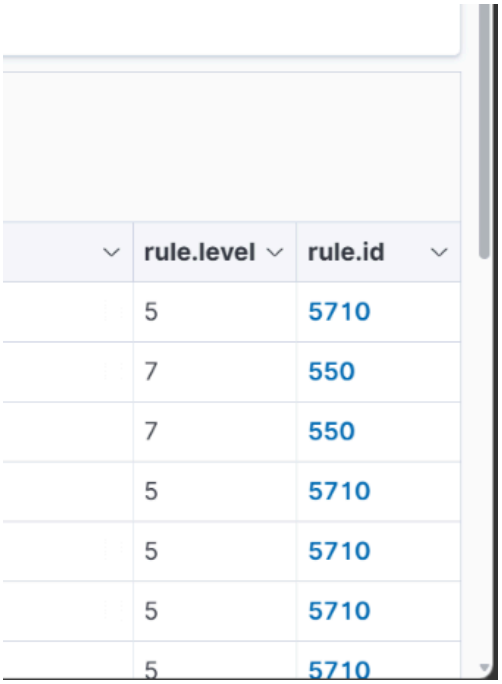
Cara Kerja dan Konfigurasi Modul SCA: How to Configure SCA

Daftar *Policies* CIS Bawaan Wazuh: Available SCA Policies

3. Ancaman Dinamis / *Dynamic Threat Score* (T), Data Log Serangan Wazuh

Ini adalah inti dari *scoring* yang akan membuat grafiknya naik-turun tiap beberapa jam secara *real-time*. Wazuh mengeluarkan *alert* dengan level 1 hingga 15. Kita kelompokkan *alert* yang muncul dalam rentang waktu tertentu (misal: per 4 jam) dan beri bobot.





- Level 1-4 (Rendah) = Bobot 1
- Level 5-7 (Menengah) = Bobot 5
- Level 8-11 (Tinggi) = Bobot 10
- Level 12-15 (Kritikal) = Bobot 25

langkah untuk melihat levelnya:

1. Masuk ke bagian **Events** (seperti langkah sebelumnya, misal dengan mengklik angka *Medium severity* atau lewat menu *Threat Hunting* > *Events*).
2. Klik tanda di sebelah kiri salah satu baris log untuk membuka detailnya.
3. Saat detailnya terbuka (yang menampilkan banyak baris *Key-Value*), perhatikan bagian atas kotak detail tersebut.
4. Kamu akan melihat dua *tab* kecil: **Table** dan **JSON**.
5. Klik *tab* **JSON**.
6. Layar akan menampilkan struktur kode JSON kurung kurawal { ... } secara utuh untuk log tersebut.

backend Node.js/Next.js menarik data dari API Wazuh, wujud datanya bukan tabel, melainkan teks format JSON. Angka level 0-16 ini bersembunyi di dalam object "rule". Contoh potongan asli log JSON dari Wazuh saat terjadi serangan *Brute Force* (gagal login berkali-kali):

Table JSON


```
{
  "_index": "wazuh-alerts-4.x-2026.03.12",
  "_id": "TC0Y4pwBbC8qr8z4rqPS",
  "_score": null,
  "_source": {
    "predecoder": {
      "hostname": "agent",
      "program_name": "sshd",
      "timestamp": "Mar 12 12:49:47"
    },
    "input": {
      "type": "log"
    },
```

```
    "agent": {
      "ip": "10.0.0.4",
      "name": "agent",
      "id": "001"
    },
    "data": {
      "srcuser": "ubuntu",
      "srcip": "146.190.83.66",
      "srcport": "47622"
    },
    "manager": {
      "name": "manager"
    },
    "rule": {
      "mail": false,
      "level": 5,
      "hipaa": [
        "164.312.b"
      ],
```


Skor ancaman pada jam ini (T{baru}) adalah total penjumlahan *alert* dikali bobotnya. Agar grafiknya bisa "turun" (pulih) jika bank sedang aman, kita gunakan rumus **Time Decay** (Penyusutan):

T{sekarang} = T{baru} + (T{sebelumnya} * a)

- "**a**" adalah faktor penyusutan, misalnya **0.5**. Artinya, dampak serangan 4 jam lalu hanya tersisa 50% di jam ini, dan akan terus mengecil jika tidak ada serangan baru.
- *Catatan:* Batasi nilai maksimal T{sekarang} di angka 100 agar kalkulasi akhir tidak *overload* (T = min(T{sekarang}, 100)).

A. Klasifikasi Level Aturan (*Rules Classification*) di Wazuh

Wazuh mengategorikan setiap *event* atau log ke dalam skala **0 hingga 16** (walaupun pada praktiknya, level 15 adalah level ancaman tertinggi untuk aturan bawaan/standar). Secara *default*, Wazuh hanya akan membunyikan *alert* dan menyimpannya di log utama jika *event* tersebut menyentuh **Level 3 ke atas**.

Level	Title	Description
0	Ignored	No action taken. Used to avoid false positives. These rules are scanned before all the others, include events with no security relevance and do not appear in the security event dashboard.
2	System low priority notification	System notification or status messages. These have no security relevance and do not appear in the security event dashboard.
3	Successful/Authorized events	These include successful login attempts, firewall allow events, etc.
4	System low priority error	Errors related to bad configurations or unused devices/applications. These have no security relevance and are usually caused by default installations or software testing.
5	User generated error	These include missed passwords, denied actions, etc. By themselves, these have no security relevance.
6	Low relevance attack	These indicate a worm or a virus that has no effect on the system (like code red for Apache servers, etc). These also include frequent IDS events and frequent errors.

7	"Bad word" matching	These include words like "bad", "error", etc. These events are most of the time unclassified and may have some security relevance.
8	First time seen	Include first time seen events. First time an IDS event is fired or the first time a user logs in. It also includes security relevant actions such as the activation of a sniffer or similar activities.
9	Error from invalid source	Include attempts to login as an unknown user or from an invalid source. May have security relevance (especially if repeated). These also include errors regarding the "admin" (root) account.
10	Multiple user generated errors	These include multiple bad passwords, multiple failed logins, etc. These may indicate an attack or simply signal that a user has forgotten their credentials.
11	Integrity checking warning	These include messages regarding the modification of binaries or the presence of rootkits (by Rootcheck). These may indicate a successful attack. Also included IDS events that will be ignored (high number of repetitions).
12	High importance event	These include error or warning messages from the system, kernel, etc. These may indicate an attack against a specific application.
13	Unusual error (high importance)	It matches a common attack pattern most of the time.
14	High importance security event	It is triggered with correlation most of the time, and it indicates an attack.
15	Severe attack	No chances of false positives. Immediate attention is necessary.

Berikut adalah pemetaan resmi dari Wazuh yang bisa dikelompokkan untuk memberikan bobot (w) di *backend*:

- **Rendah (Level 0 - 4) | *Bobot: 1***

Ini adalah aktivitas normal atau *noise* sistem.

  - *Level 0:* Diabaikan (*Ignored*).
  - *Level 3:* Aktivitas sukses/resmi (*Successful/Authorized events*), seperti staf yang berhasil *login* atau *firewall* yang mengizinkan koneksi.
  - *Level 4:* Error sistem prioritas rendah (*System low priority error*).

- **Menengah (Level 5 - 7) | *Bobot: 5***

Ini adalah aktivitas anomali ringan atau *error* dari pengguna.

  - *Level 5: User generated error* (misal: salah ketik *password* satu atau dua kali).
  - *Level 7:* Pencocokan kata buruk (*"Bad word" matching*), seperti ada log sistem yang mengandung kata "error" atau "fatal".

- **Tinggi (Level 8 - 11) | *Bobot: 10***

Ini adalah indikasi awal serangan atau perilaku yang sangat mencurigakan.

  - *Level 8: First time seen* (misal: pengguna *login* untuk pertama kalinya dari IP yang tidak pernah dilihat sebelumnya).
  - *Level 10: Multiple user generated errors* (misal: indikasi serangan *Brute Force* karena ada puluhan kali gagal *login* berturut-turut).
  - *Level 11: Integrity checking warning* (peringatan integritas dari modul FIM/Rootcheck, misal: ada *file* sistem yang tiba-tiba dimodifikasi).

- **Kritikal (Level 12 - 15) | *Bobot: 25***

Ini adalah serangan nyata, ancaman tingkat tinggi, atau indikasi sistem sudah diretas.

  - *Level 12-14: High importance security event* (indikasi serangan spesifik terhadap aplikasi atau peretasan yang terkonfirmasi).
  - *Level 15: Severe attack* (Serangan parah yang tidak mungkin *false positive*, seperti deteksi *malware* aktif atau *rootkit*). Butuh penanganan segera!

B. Implementasi ke Formula *Dynamic Threat Score* (T)

Di *backend*, tidak perlu mengevaluasi *alert* ini satu per satu secara *real-time* tiap detik karena bisa membebani *server*. Cukup melakukan *query* ke basis data Wazuh (Wazuh Indexer / OpenSearch) setiap periode tertentu, misalnya **setiap 1 jam atau 4 jam**.

Logikanya akan seperti ini:

1. *Backend* menarik data: "Berapa banyak *alert* yang terjadi di IP Server Database dalam 4 jam terakhir, dikelompokkan berdasarkan Level?"
2. Misal hasilnya: Ada 50 *alert* Level 3 (Sukses Login), 5 *alert* Level 10 (Brute Force), dan 0 *alert* Level 15.
3. Hitung T{baru} dengan rumus akumulasi bobot:
   T{baru} = ({Jumlah Lvl 1-4} * 1) + ({Jumlah Lvl 5-7} * 5) + ({Jumlah Lvl 8-11} * 10) +

({Jumlah Lvl 12-15} * 25)
T{baru} = (50 * 1) + (0 * 5) + (5 * 10) + (0 * 25)
T{baru} = 50 + 0 + 50 + 0 = 100

4. Menghitung Skor Ancaman Final (T{sekarang}) dengan Time DecayRumus utamanya adalah:T{sekarang} = T{baru} + (T{sebelumnya} * a)
5. **Periode 1: Pukul 08:00 - 12:00** (Terjadi Serangan Brute Force) Backend menghitung log Wazuh. Seperti hasil di Langkah 3, didapat T{baru} = 100.

   Karena 4 jam sebelumnya kondisi bank aman, maka T{sebelumnya} = 0.

   Kalkulasi Time Decay: T{sekarang} = 100 + (0 * 0.5) = 100

   Hasil di Dashboard: Skor ancaman aset tersebut melompat ke angka 100 (Kritis/Merah).

6. **Periode 2: Pukul 12:00 - 16:00** (Serangan Berhenti, tapi Tim SOC masih Waspada)Sistem backend menarik log Wazuh lagi. Penyerang sudah diblokir, sehingga log yang muncul hanya noise biasa (misal beberapa login sukses dan error ketik biasa). Backend menghitung rumusnya dan mendapat T{baru} = 10.

   Sistem mengambil skor dari Periode 1 untuk dijadikan sisa ancaman, jadi T{sebelumnya} = 100.

   Kalkulasi Time Decay: T{sekarang} = 10 + (100 * 0.5) = 10 + 50 = 60

   Hasil di Dashboard: Skor turun menjadi 60 (Waspada/Kuning). Grafik mulai melandai. Walaupun sistem saat ini nyaris aman (T{baru} cuma 10), skor di dashboard tidak langsung anjlok ke 10 karena "efek sisa" kepanikan serangan brute force sebelumnya masih diperhitungkan.

7. **Periode 3: Pukul 16:00 - 20:00** (Kondisi Makin Normal)Backend menarik log lagi. Kondisi masih normal, aktivitas alert sangat sedikit sehingga T{baru} = 4.

   Sistem mengambil skor dari Periode 2, jadi T{sebelumnya} = 60.

   Kalkulasi Time Decay: T{sekarang} = 4 + (60 * 0.5) = 4 + 30 = 34

   Hasil di Dashboard: Skor makin turun menjadi 34 (Menuju Aman/Hijau).

Catatan: Bobot (1, 5, 10, 25) di atas adalah contoh angka yang bisa dimodifikasi (tuning) nanti saat uji coba simulasi agar grafiknya tidak terlalu cepat menyentuh batas maksimal.

Wazuh Rules Classification (Level 0 - 16): Rules classification - Wazuh Documentation

Format Penulisan Aturan dan Level: Rules Syntax - Wazuh Documentation

4. Formula Akhir *Risk Score* (R)

Sekarang kita gabungkan semuanya di *backend* untuk mendapatkan skor final (0-100) per aset:

$$R = I * (w1*V + w2*T)$$

- "**w1**" dan "**w2**" adalah rasio bobot antara Kerentanan (Sistem bawaan) dan Ancaman (Serangan aktif). Totalnya harus 1.
- Misalnya, set w1 = 0.3 (30% bobot dari CIS) dan w2 = 0.7 (70% bobot dari *alert* aktif yang lebih berbahaya).

Contoh Simulasi Singkat:

Server *Database* (Likert 5 –> I = 1.0). CIS Lulus 80% (V = 20). Sedang ada *brute force attack* level 10 (T = 60).

R = 1.0 * ((0.3 * 20) + (0.7 * 60))
R = 1.0 * (6 + 42) = 48
Skor Risiko Aset = 48 / 100 (Sedang menuju ke arah bahaya).

5. Adaptasi Skala CVSS v3.1 / v4.0

CVSS adalah standar global yang digunakan oleh seluruh praktisi *security* (termasuk Wazuh dalam menilai kerentanan) untuk mengukur tingkat keparahan. CVSS menggunakan skala **0.0 hingga 10.0**.

Karena target *dashboard* manajemen kita menggunakan skala persentase **0 hingga 100**, kita cukup **mengalikan skala CVSS dengan 10** sebagai *baseline* (nilai dasar) ambang batasnya.

Revisi Rentang Skor Berdasarkan Standar CVSS:

- **Low (Skor: 0 - 39):** Setara dengan CVSS 0.0 - 3.9.
- **Medium (Skor: 40 - 69):** Setara dengan CVSS 4.0 - 6.9.
- **High (Skor: 70 - 89):** Setara dengan CVSS 7.0 - 8.9.
- **Critical (Skor: 90 - 100):** Setara dengan CVSS 9.0 - 10.0.

Pernyataan Argumen di Laporan: *"Meskipun sistem ini menghasilkan skor dinamis dari telemetri Wazuh, pengelompokan tingkat keparahan (Severity Ratings) pada dashboard diadaptasi dari standar kualitatif Common Vulnerability Scoring System (CVSS) v3.1. Skala CVSS 0.0 - 10.0 dikonversi menjadi persentase 0 - 100 untuk memudahkan pembacaan oleh level eksekutif (decision maker) perbankan, sehingga rentang yang dihasilkan memiliki validitas standar industri."*

- **Tautan Resmi CVSS untuk Daftar Pustaka:** CVSS v3.1 Specification Document - Qualitative Severity Rating Scale

Catatan Penting Implementasi (Disclaimer)

Formulasi *Dynamic Cyber Risk Scoring* yang diusulkan pada tahap ini merupakan **model dasar (*baseline*)** yang dirancang khusus untuk kebutuhan *Proof of Concept* (POC) dan belum berada pada tahap optimal untuk lingkungan produksi.

Sistem penilaian risiko siber pada dasarnya cross data telemetri teknis (Wazuh) dengan prioritas bisnis, sehingga akurasinya sangat bergantung pada konteks organisasi. Agar *dashboard* ini dapat memberikan nilai tambah yang nyata bagi *decision maker*, terdapat beberapa tindak lanjut yang wajib dilakukan:

- **Kalibrasi dan Uji Coba Berkelanjutan (*Trial & Error*):** Bobot ancaman (*threat weight*), parameter kerentanan, dan faktor penyusutan waktu (*time decay*) di dalam algoritma masih menggunakan nilai asumsi. Diperlukan simulasi dengan data log Wazuh yang sesungguhnya untuk melihat apakah grafik yang dihasilkan terlalu sensitif (mudah naik) atau terlalu lambat merespons anomali.
- **Validasi Prioritas Aset dengan Manajemen:** Nilai dampak bisnis (Skala Likert) tidak bisa ditentukan oleh tim teknis secara sepihak. Diperlukan komunikasi dua arah yang intensif dengan pihak manajemen dan *stakeholder* terkait (seperti pengguna konsep *Balanced Scorecard*) untuk memetakan aset mana yang paling kritikal bagi operasional perbankan.
- **Penyesuaian Ambang Batas (*Threshold*):** Manajemen perlu mendefinisikan dengan jelas di titik mana sebuah skor (0-100) dianggap "Aman", "Waspada", atau "Kritis", serta bagaimana prosedur tindak lanjut (*Standard Operating Procedure*) ketika skor menyentuh angka tertentu.

Prototipe ini berfungsi sebagai pemantik diskusi antara tim SOC (Keamanan Siber) dan jajaran Eksekutif, bukan sebagai produk akhir yang langsung menggantikan proses audit keamanan yang sudah berjalan.